GalaxEye — ML Systems Take-Home Assignment

Role: Backend Engineer, ML Systems | **Candidate:** Anupam Patil | **Deployment Target:** Air-Gapped Edge Hardware

---

1. System Design & Offline Edge Architecture (Part 1)

At GalaxEye, satellite sensor imagery must be processed, classified, and indexed on isolated hardware with zero internet connectivity. Under these constraints, reliance on cloud-hosted inference APIs, external message brokers, or managed object storage is eliminated. The architecture is designed as a self-contained, crash-resilient edge service prioritizing deterministic CPU throughput, multi-factor uncertainty detection, and low operational overhead.

End-to-End Tile Lifecycle:

- **Ingestion & Deduplication:** Tiles arrive individually (/classify) or in downlink bursts (/batch). A cryptographic SHA-256 hash is computed immediately over the raw byte stream. If the hash exists in the catalog, inference is bypassed and the cached record is returned instantly.
- **Validation & Preprocessing:** Decoding, dimension verification (64x64 EuroSAT standard), 3-band RGB formatting, and channel normalization using Sentinel-2 optical parameters.
- **Dual-Runtime Inference:** Dispatched primarily to **ONNX Runtime CPU** with SIMD graph optimizations, operating at **2.66 ms / tile** (~380 tiles/sec) with transparent PyTorch CPU fallback.
- **Multi-Factor Uncertainty Gating:** Softmax confidence is complemented by *Prediction Margin* (p1 - p2) and *Normalized Shannon Entropy*. Tiles failing criteria (Conf < 0.60, Margin < 0.20, or Entropy > 0.65) are flagged UNCERTAIN.
- **Dual-Tier Local Persistence:** Raw rasters are stored in a content-addressed directory (storage/<sha256>.png), while metadata and full probability vectors are committed to SQLite with Write-Ahead Logging (WAL).
- **Analyst Query & Active Learning:** Exposed via REST API and an embedded offline dashboard (/dashboard) allowing analysts to filter by class/uncertainty and record ground-truth corrections into an audit table.

Architectural Trade-offs & Decisions:

Decision Area	Options Considered	Chosen Approach & Rationale
Uncertainty	• Hard drop < 60% • Silent argmax	**Soft Probabilities + 3-Factor Tagging:** Preserves probability vectors; routes ambiguous border tiles (Highway vs River) to review queue without data loss.
Storage Model	• DB BLOB storage • PostgreSQL + PostGIS	**Hybrid Filesystem + SQLite WAL:** Eliminates DB cache bloat; WAL enables concurrent reads during writes; zero external daemon dependencies.
Downlink Throughput	• Single HTTP POST • Distributed queue (Celery)	**Vectorized Micro-Batching:** Ingests multiple tiles in one forward pass using CPU SIMD instructions, achieving ~380 tiles/sec without broker overhead.

2. Implementation & Empirical Benchmark (Part 2)

A lightweight convolutional neural network was trained locally on the 1,050 EuroSAT candidate tiles and exported to ONNX format. The model was evaluated against all **210 unseen test tiles** in the evaluation set using ground-truth labels from eval_labels.csv.

Metric	Observed Result	Operational Implication
Overall Accuracy	**84.76%** (178 / 210)	Solid baseline for 64x64 single crops without spatial context.
Inference Engine	**ONNX Runtime CPU**	Graph optimization enabled; zero GPU requirement.
Mean Latency	**2.66 ms / tile**	Throughput of ~375–385 tiles/sec on standard CPU.
Mean Shannon Entropy	**0.227** [0 to 1]	High certainty across unambiguous terrestrial zones.
Uncertainty Flags	**28 tiles (13.3%)**	Confusions cluster on Highway vs River linear features.
Test Suite Pass Rate	**11 / 11 tests passed**	Verifies health, batching, active learning, and edge cases.

3. Problem Solving & Operational Scenarios (Part 3)

Q1: Classifier is wrong ~30% of the time. What do you do, and how to decide if it's 'good enough'?

Immediate Steps: Rather than reacting to a global 30% error metric, I immediately inspect the per-class confusion matrix. In satellite imagery, errors typically cluster along physically plausible ambiguities (e.g. Highways vs Rivers, both being narrow linear features, or Crops vs Forest during seasonal changes), rather than catastrophic errors (e.g. Industrial vs SeaLake). Next, I calibrate the uncertainty decision boundary. In calibrated networks, misclassifications concentrate in the high-entropy / low-margin band. By routing samples with Confidence < 0.60 or Entropy > 0.65 to the UNCERTAIN review queue, effective accuracy on the automated subset rises to >90%, while human analysts focus on the ambiguous 10–15%. Finally, I evaluate test-time augmentation (4-way rotation averaging), which is computationally cheap and physically invariant for nadir satellite views.
Deciding if 70% is 'Good Enough': Utility is governed by operational cost asymmetry. In wide-area disaster or flood screening, high recall (95%) with 70% precision is immensely valuable because it eliminates 99% of empty terrain and reduces human scanning from millions of km² to a focused triage queue. Conversely, for autonomous navigation or targeting, 70% is unacceptable without human-in-the-loop verification.

Q2: Service runs offline with no live monitoring. A month later, how to know it's working?

In an air-gapped system, telemetry must be local, autonomous, and self-verifying:
1. **Structured Local Telemetry & Log Compaction:** Record every inference event with timestamp, tile hash, latency, confidence, entropy, and memory RSS. Rotating ring-buffer logs (500MB cap) prevent storage exhaustion.
2. **Statistical Drift Detection:** A daily offline cron computes rolling 7-day distribution statistics. Alert on: (a) *Class Collapse* (e.g. Forest suddenly reaching 98%, indicating lens obstruction or sensor saturation); (b) *Entropy Surge* (mean entropy rising from 0.22 to 0.70, indicating optical defocus or atmospheric haze); (c) *Latency Spikes* (indicating thermal throttling or storage lock contention).
3. **Canary Validation ('Golden Set'):** A cryptographically signed suite of 10 reference tiles is stored in firmware. Daily autonomous re-inference tests against known ground truth. If predictions diverge by >1%, a hardware failure flag is tripped.
4. **Analyst Maintenance Extraction:** When field personnel connect via local service port, a single command generates an encrypted diagnostic tarball containing rolling histograms, database WAL status, and error logs.

Q3: Tiles arrive fine, but stored results look wrong. Walk through your diagnostic steps in order.

When inputs arrive normally but outputs are erroneous, the defect lies in preprocessing, model inference, or serialization:
• **Step 1 (Isolate & Reproduce):** Retrieve the exact raw tile binary from storage/<sha256>.png and run it through a standalone offline CLI script to rule out HTTP server issues.
• **Step 2 (Channel Order & Bit Depth):** Verify color channel alignment. OpenCV loads BGR while PIL loads RGB. If channels are inverted, water is classified as vegetation. Verify if 16-bit satellite GeoTIFFs were naively downcast to 8-bit, clipping pixel ranges.
• **Step 3 (Normalization Bounds):** Inspect tensor statistics (min, max, mean). Confirm input scaling matches training parameters. If unnormalized [0, 255] pixels are fed, ReLU activations saturate and logits collapse.
• **Step 4 (Class Index Mapping):** Check dictionary ordering. PyTorch ImageFolder sorts alphabetically; if the inference script or evaluation ground truth uses an alternate order, outputs are consistently mapped to incorrect labels.
• **Step 5 (Model State & Checksum):** Verify SHA-256 checksums of classifier.onnx and classifier.pt to ensure no binary corruption during deployment. Verify model.eval() is active.
• **Step 6 (Database Serialization):** Inspect raw in-memory dictionaries against the committed SQLite row to ensure parameter bindings did not swap column values.

Q4: What is the weakest part of your design, and what would break it first?

The Weakest Link: Synchronous disk I/O and unbuffered single-writer SQLite transactions during burst downlinks.
What Breaks First: Satellite ground passes transmit thousands of tiles in brief 3–5 minute windows. While ONNX CPU inference takes only 2.66 ms, writing each raw tile synchronously to disk and committing to SQLite WAL mode creates heavy random write I/O. On edge hardware using industrial flash/eMMC storage with low random write IOPS, disk write queues will saturate, driving latency above 200 ms and causing connection timeouts. Furthermore, while SQLite WAL allows concurrent readers, it enforces a single writer; concurrent ingestion threads will encounter database is locked.
Production Hardening: Decouple ingestion from persistence using an in-memory ring buffer (asyncio.Queue), batch database writes into transactions of 100–500 rows (executemany), and enforce an automated LRU disk quota policy.

4. Questions for the GalaxEye Engineering Team

1. **Multispectral & SAR Payloads:** Will future pipelines ingest 10–13 band multispectral or Synthetic Aperture Radar (SAR) imagery? If so, what radiometric calibration and speckle filtering should be incorporated into edge preprocessing?
2. **Geospatial Referencing:** Are incoming tiles georeferenced via GeoTIFF metadata or sidecar files? (This determines whether SpatiaLite indexing should be enabled for spatial bounding-box queries).
3. **Active Learning Sync:** How will verified ground-truth corrections captured in audit_corrections be synchronized back to central mission control for offline active learning?